

Binární stromy

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

03.06.2026

CC BY-NC-SA 4.0

Binární strom

Co je binární strom?

Minule jsme se seznámili s obecnými stromy, kde uzel mohl mít libovolný počet potomků

Co je binární strom?

Minule jsme se seznámili s obecnými stromy, kde uzel mohl mít libovolný počet potomků

V praxi se ale nejčastěji setkáváme se stromy, kde má každý uzel

Co je binární strom?

Minule jsme se seznámili s obecnými stromy, kde uzel mohl mít libovolný počet potomků

V praxi se ale nejčastěji setkáváme se stromy, kde má každý uzel **maximálně dva potomky**

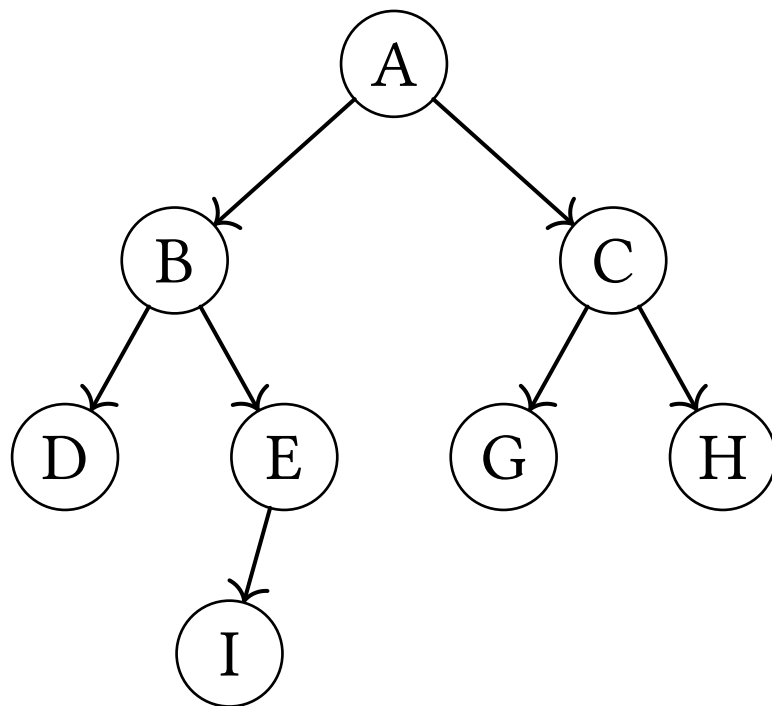
Co je binární strom?

Minule jsme se seznámili s obecnými stromy, kde uzel mohl mít libovolný počet potomků

V praxi se ale nejčastěji setkáváme se stromy, kde má každý uzel **maximálně dva potomky**

Těmto potomkům říkáme **levý** a **pravý**

Co je binární strom?



Typy binárních stromů

Typy binárních stromů

U **plného** binárního stromu (*full*) má každý uzel buď **0**, nebo **2** potomky

Typy binárních stromů

U **plného** binárního stromu (*full*) má každý uzel buď **0**, nebo **2** potomky

U **úplného** binárního stromu (*complete*) jsou všechny úrovně plné, kromě poslední, která je zaplněna **zleva**

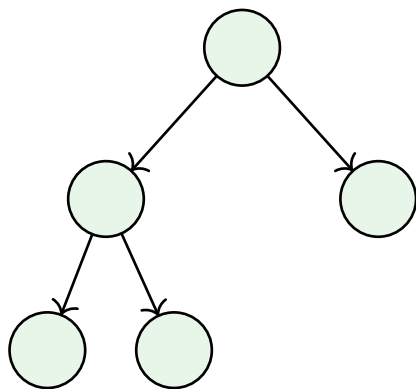
Typy binárních stromů

U **plného** binárního stromu (*full*) má každý uzel buď **0**, nebo **2** potomky

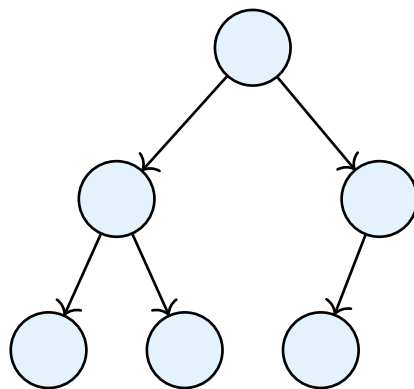
U **úplného** binárního stromu (*complete*) jsou všechny úrovně plné, kromě poslední, která je zaplněna **zleva**

U **perfektního** binárního stromu (*perfect*) mají všechny vnitřní uzly 2 potomky a všechny listy jsou na **stejné úrovni**

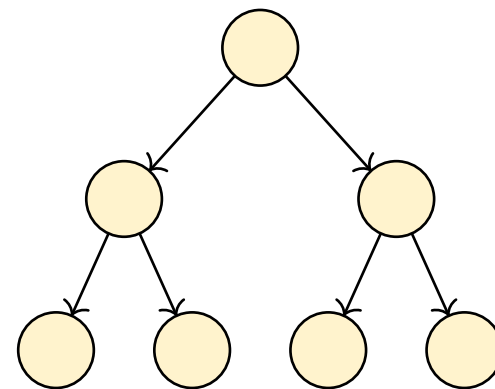
Typy binárních stromů



Full



Complete



Perfect

Implementace v C

Obecný strom

Obecný strom

U obecného stromu má uzel **libovolný** počet potomků:

Obecný strom

U obecného stromu má uzel **libovolný** počet potomků:

```
typedef struct TreeNode {  
    int data;  
    struct TreeNode **children;  
    int child_count;  
} TreeNode;
```


Obecný strom

U obecného stromu má uzel **libovolný** počet potomků:

```
typedef struct TreeNode {  
    int data;  
    struct TreeNode **children;  
    int child_count;  
} TreeNode;
```

Nevýhodou je, že musíme dynamicky alokovat pole potomků a správa paměti je složitější

Binární strom

Binární strom

U binárního stromu má uzel maximálně **dva** potomky:

Binární strom

U binárního stromu má uzel maximálně **dva** potomky:

```
typedef struct Node {  
    int data;  
    struct Node *left;  
    struct Node *right;  
} Node;
```

Binární strom

U binárního stromu má uzel maximálně **dva** potomky:

```
typedef struct Node {  
    int data;  
    struct Node *left;  
    struct Node *right;  
} Node;
```

Mnohem jednodušší!

Binární strom

U binárního stromu má uzel maximálně **dva** potomky:

```
typedef struct Node {  
    int data;  
    struct Node *left;  
    struct Node *right;  
} Node;
```

Mnohem jednodušší! Podobná struktura jako u spojového seznamu, jen místo jednoho next máme

Binární strom

U binárního stromu má uzel maximálně **dva** potomky:

```
typedef struct Node {  
    int data;  
    struct Node *left;  
    struct Node *right;  
} Node;
```

Mnohem jednodušší! Podobná struktura jako u spojového seznamu, jen místo jednoho next máme left a right

Binární strom

Vytvoření jednoduchého stromu:

Binární strom

Vytvoření jednoduchého stromu:

```
Node *create_node(int data) {  
    Node *node = (Node *) malloc(sizeof(Node));  
    node->data = data;  
    node->left = NULL; node->right = NULL;  
    return node;  
}
```

Binární strom

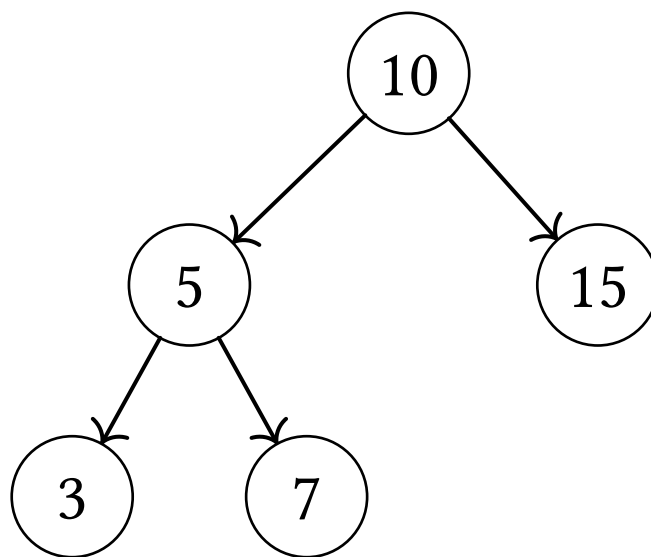
Vytvoření jednoduchého stromu:

```
Node *create_node(int data) {  
    Node *node = (Node *) malloc(sizeof(Node));  
    node->data = data;  
    node->left = NULL; node->right = NULL;  
    return node;  
}
```

```
Node *root = create_node(10);  
root->left = create_node(5);  
root->right = create_node(15);  
root->left->left = create_node(3);  
root->left->right = create_node(7);
```

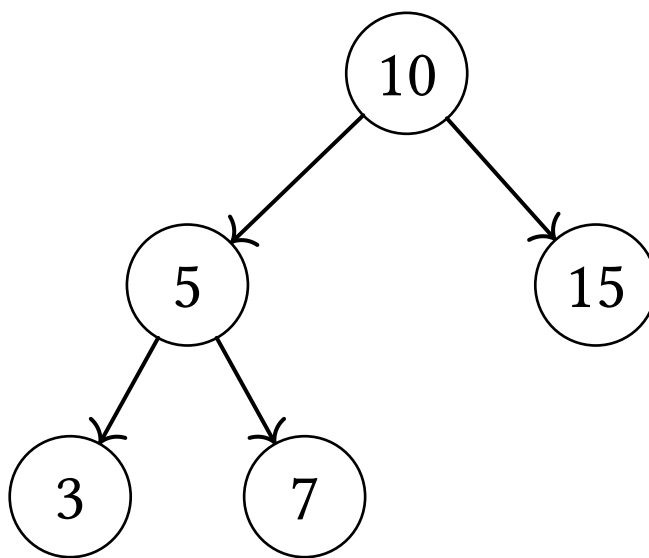
Binární strom

Výsledek:



Binární strom

Výsledek:



Všimněte si, že uzly 15, 3 a 7 mají `left == NULL` a `right == NULL`

Průchody binárním stromem

Proč průchody?

Proč průchody?

Jak **systematicky navštívit** všechny uzly stromu?

Proč průchody?

Jak **systematicky navštívit** všechny uzly stromu?

Na rozdíl od seznamu nemůžeme jít jen „dopředu“, musíme se rozhodnout:

Proč průchody?

Jak **systematicky navštívit** všechny uzly stromu?

Na rozdíl od seznamu nemůžeme jít jen „dopředu“, musíme se rozhodnout: jít **doleva**, **doprava**, nebo **zpracovat** aktuální uzel?

Proč průchody?

Jak **systematicky navštívit** všechny uzly stromu?

Na rozdíl od seznamu nemůžeme jít jen „dopředu“, musíme se rozhodnout: jít **doleva**, **doprava**, nebo **zpracovat** aktuální uzel?

Tři základní způsoby se liší **pořadím** těchto operací:

Proč průchody?

Jak **systematicky navštívit** všechny uzly stromu?

Na rozdíl od seznamu nemůžeme jít jen „dopředu“, musíme se rozhodnout: jít **doleva**, **doprava**, nebo **zpracovat** aktuální uzel?

Tři základní způsoby se liší **pořadím** těchto operací:

Průchod	Zkratka	Pořadí
Inorder		

Proč průchody?

Jak **systematicky navštívit** všechny uzly stromu?

Na rozdíl od seznamu nemůžeme jít jen „dopředu“, musíme se rozhodnout: jít **doleva**, **doprava**, nebo **zpracovat** aktuální uzel?

Tři základní způsoby se liší **pořadím** těchto operací:

Průchod	Zkratka	Pořadí
Inorder	LNR	Levý, Uzel , Pravý

Proč průchody?

Jak **systematicky navštívit** všechny uzly stromu?

Na rozdíl od seznamu nemůžeme jít jen „dopředu“, musíme se rozhodnout: jít **doleva**, **doprava**, nebo **zpracovat** aktuální uzel?

Tři základní způsoby se liší **pořadím** těchto operací:

Průchod	Zkratka	Pořadí
Inorder	LNR	Levý, Uzel , Pravý
Preorder		

Proč průchody?

Jak **systematicky navštívit** všechny uzly stromu?

Na rozdíl od seznamu nemůžeme jít jen „dopředu“, musíme se rozhodnout: jít **doleva**, **doprava**, nebo **zpracovat** aktuální uzel?

Tři základní způsoby se liší **pořadím** těchto operací:

Průchod	Zkratka	Pořadí
Inorder	LNR	Levý, Uzel , Pravý
Preorder	NLR	Uzel , Levý, Pravý

Proč průchody?

Jak **systematicky navštívit** všechny uzly stromu?

Na rozdíl od seznamu nemůžeme jít jen „dopředu“, musíme se rozhodnout: jít **doleva**, **doprava**, nebo **zpracovat** aktuální uzel?

Tři základní způsoby se liší **pořadím** těchto operací:

Průchod	Zkratka	Pořadí
Inorder	LNR	Levý, Uzel , Pravý
Preorder	NLR	Uzel , Levý, Pravý
Postorder		

Proč průchody?

Jak **systematicky navštívit** všechny uzly stromu?

Na rozdíl od seznamu nemůžeme jít jen „dopředu“, musíme se rozhodnout: jít **doleva**, **doprava**, nebo **zpracovat** aktuální uzel?

Tři základní způsoby se liší **pořadím** těchto operací:

Průchod	Zkratka	Pořadí
Inorder	LNR	Levý, Uzel , Pravý
Preorder	NLR	Uzel , Levý, Pravý
Postorder	LRN	Levý, Pravý, Uzel

Inorder (LNR)

Inorder (LNR)

Postup:

1. Rekurzivně projdi **levý** podstrom

Inorder (LNR)

Postup:

1. Rekurzivně projdi **levý** podstrom
2. Zpracuj **aktuální uzel**

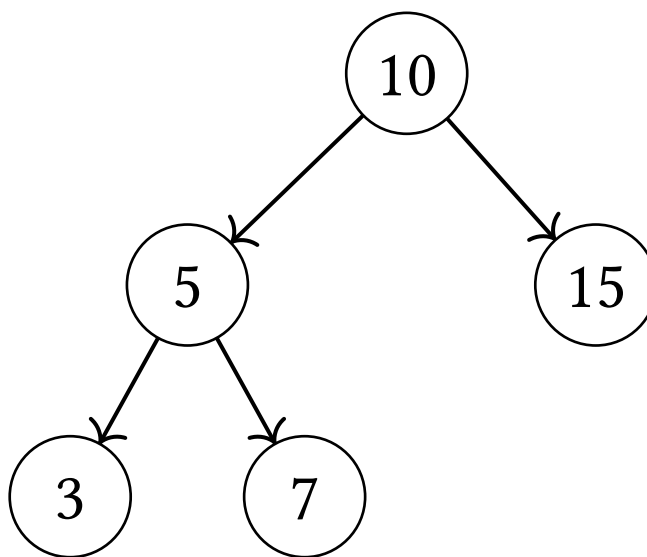
Inorder (LNR)

Postup:

1. Rekurzivně projdi **levý** podstrom
2. Zpracuj **aktuální uzel**
3. Rekurzivně projdi **pravý** podstrom

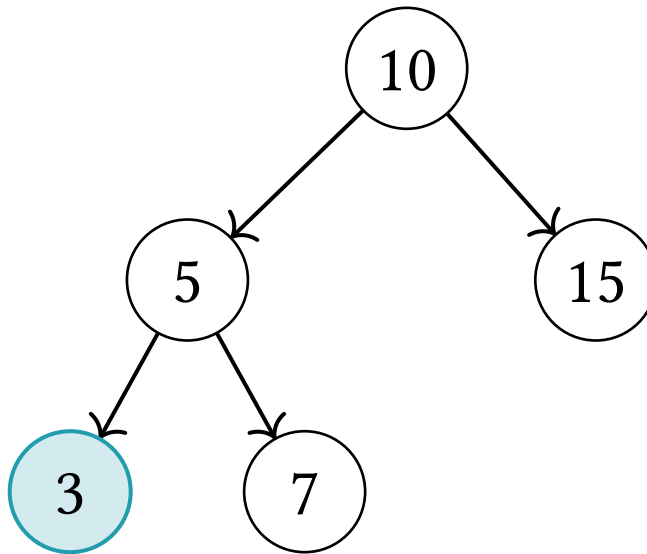
Inorder (LNR)

Inorder na stromu [10, 5, 15, 3, 7]:



Inorder (LNR)

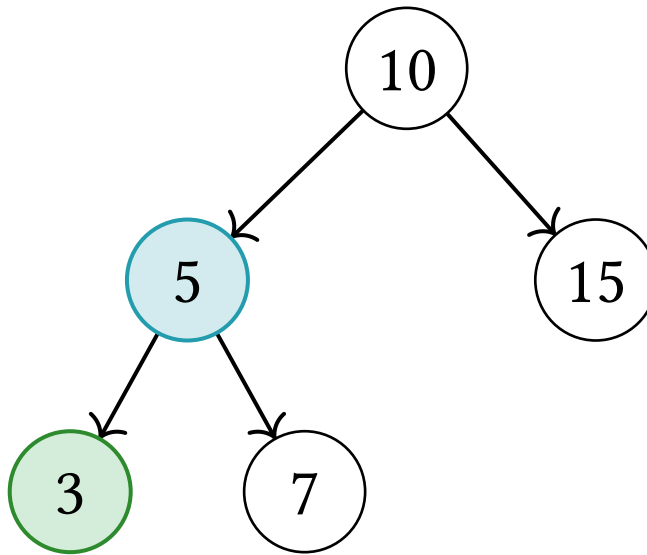
Inorder na stromu [10, 5, 15, 3, 7]:



Výstup: 3

Inorder (LNR)

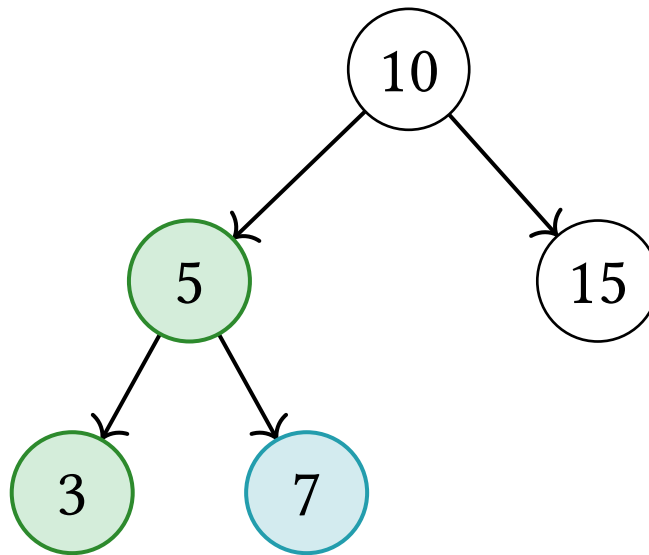
Inorder na stromu [10, 5, 15, 3, 7]:



Výstup: 3, 5

Inorder (LNR)

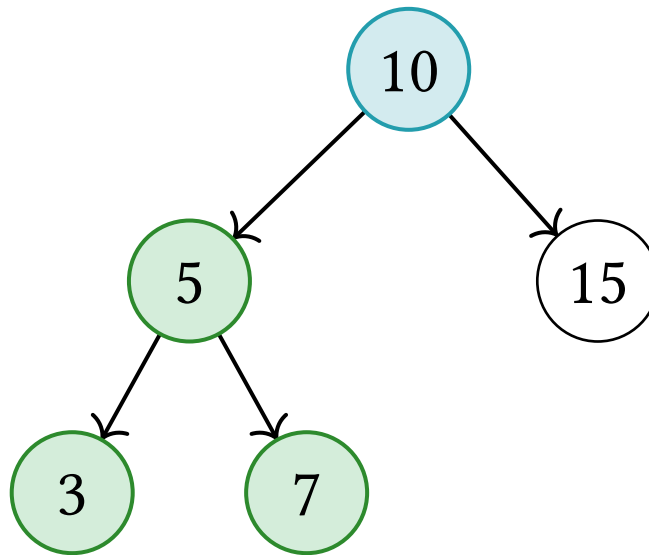
Inorder na stromu [10, 5, 15, 3, 7]:



Výstup: 3, 5, 7

Inorder (LNR)

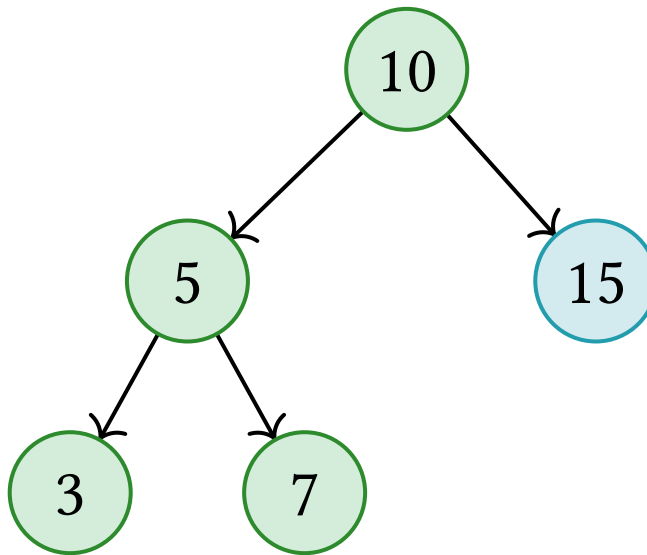
Inorder na stromu [10, 5, 15, 3, 7]:



Výstup: 3, 5, 7, 10

Inorder (LNR)

Inorder na stromu [10, 5, 15, 3, 7]:



Výstup: 3, 5, 7, 10, 15

Preorder (NLR)

Preorder (NLR)

Postup:

1. Zpracuj **aktuální uzel**

Preorder (NLR)

Postup:

1. Zpracuj **aktuální uzel**
2. Rekurzivně projdi **levý** podstrom

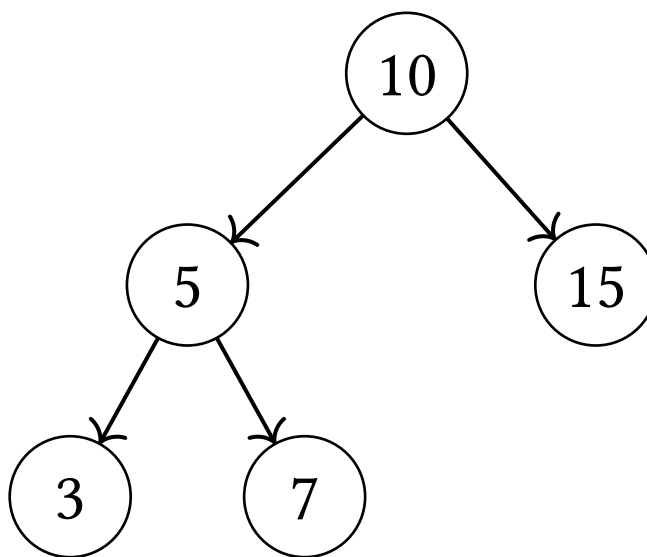
Preorder (NLR)

Postup:

1. Zpracuj **aktuální uzel**
2. Rekurzivně projdi **levý** podstrom
3. Rekurzivně projdi **pravý** podstrom

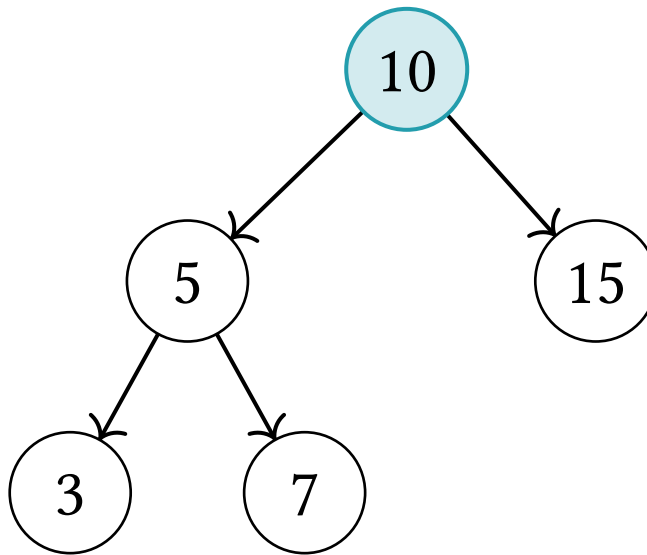
Preorder (NLR)

Preorder na stromu [10, 5, 15, 3, 7]:



Preorder (NLR)

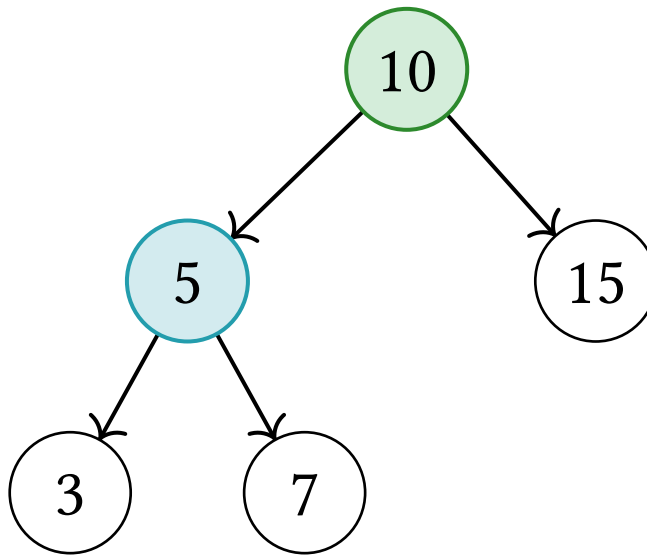
Preorder na stromu [10, 5, 15, 3, 7]:



Výstup: 10

Preorder (NLR)

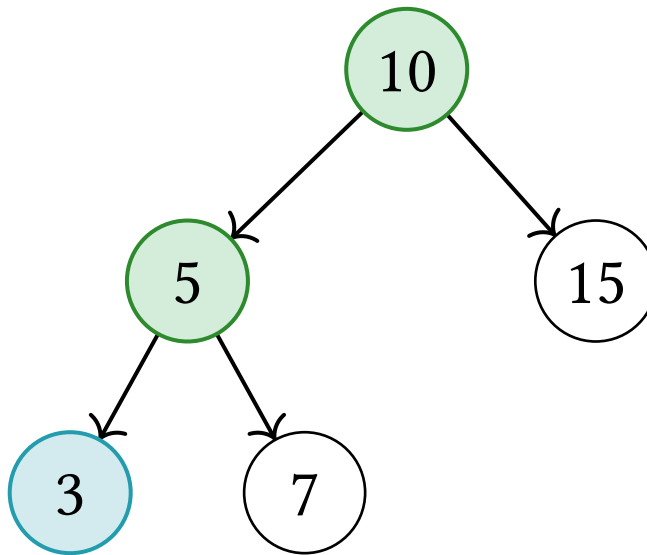
Preorder na stromu [10, 5, 15, 3, 7]:



Výstup: 10, 5

Preorder (NLR)

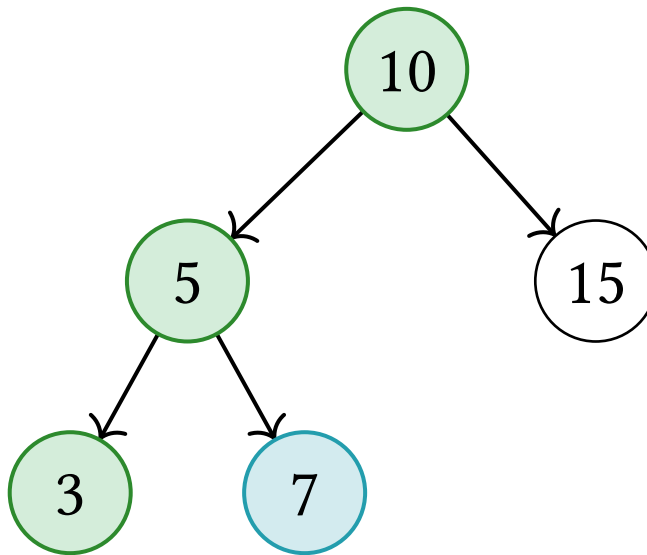
Preorder na stromu [10, 5, 15, 3, 7]:



Výstup: 10, 5, 3

Preorder (NLR)

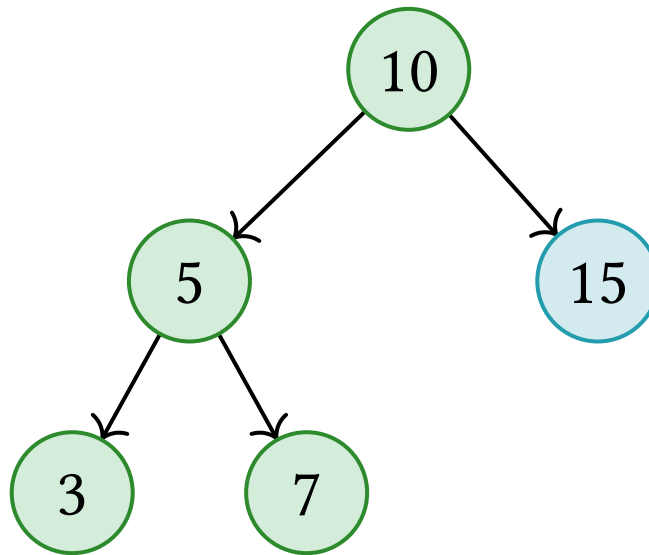
Preorder na stromu [10, 5, 15, 3, 7]:



Výstup: 10, 5, 3, 7

Preorder (NLR)

Preorder na stromu [10, 5, 15, 3, 7]:



Výstup: 10, 5, 3, 7, 15

Postorder (LRN)

Postorder (LRN)

Postup:

1. Rekurzivně projdi **levý** podstrom

Postorder (LRN)

Postup:

1. Rekurzivně projdi **levý** podstrom
2. Rekurzivně projdi **pravý** podstrom

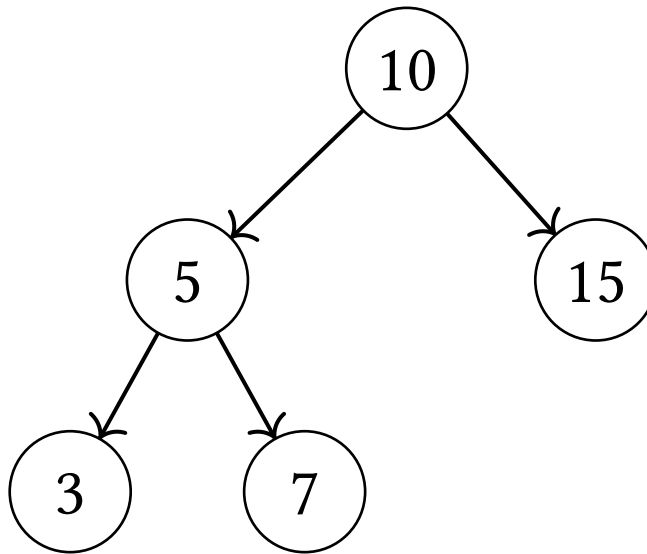
Postorder (LRN)

Postup:

1. Rekurzivně projdi **levý** podstrom
2. Rekurzivně projdi **pravý** podstrom
3. Zpracuj **aktuální uzel**

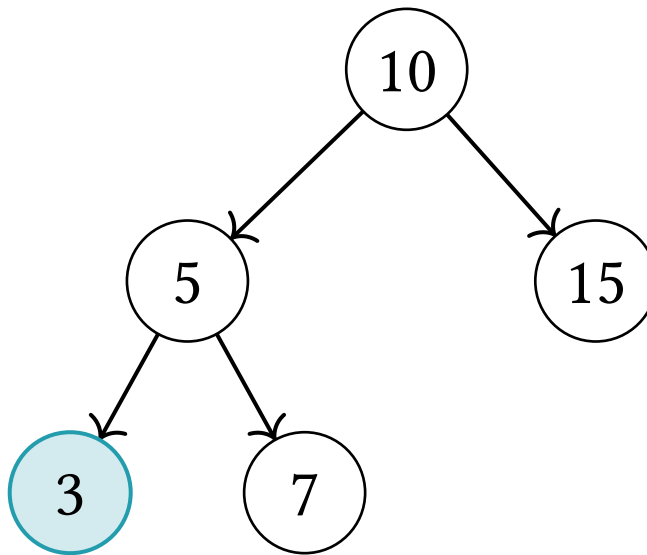
Postorder (LRN)

Postorder na stromu [10, 5, 15, 3, 7]:



Postorder (LRN)

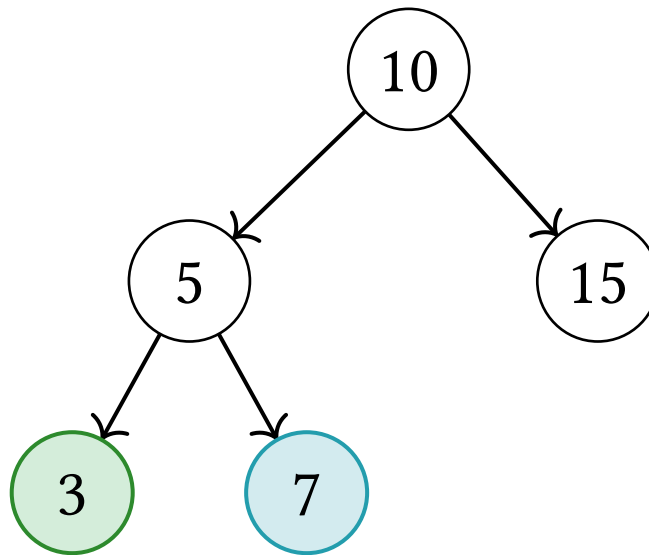
Postorder na stromu [10, 5, 15, 3, 7]:



Výstup: 3

Postorder (LRN)

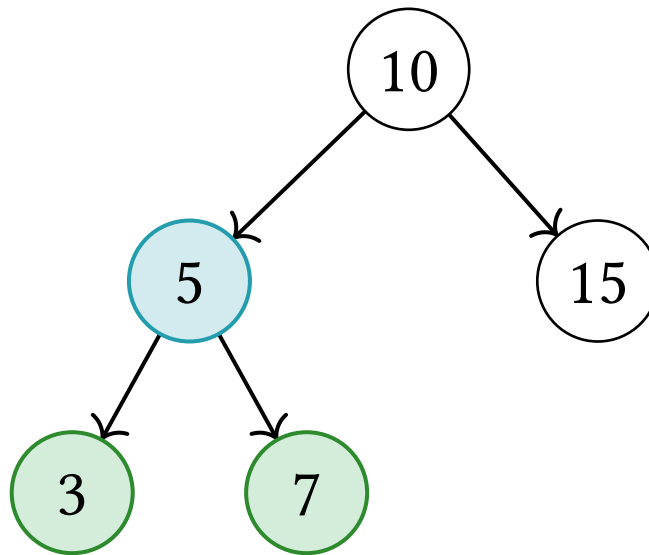
Postorder na stromu [10, 5, 15, 3, 7]:



Výstup: 3, 7

Postorder (LRN)

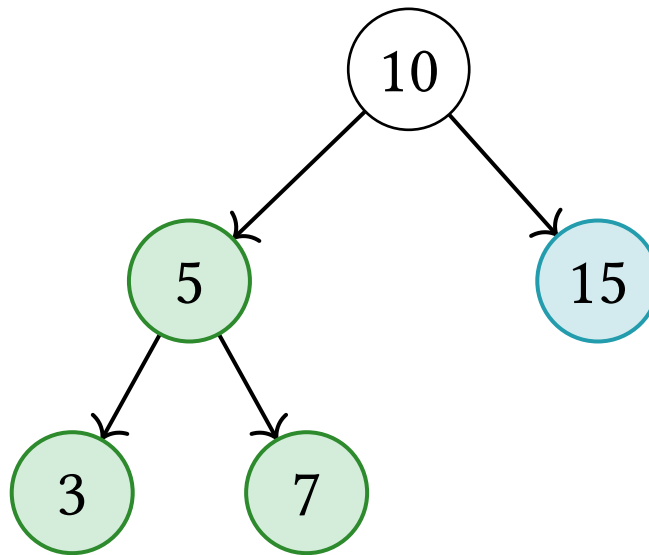
Postorder na stromu [10, 5, 15, 3, 7]:



Výstup: 3, 7, 5

Postorder (LRN)

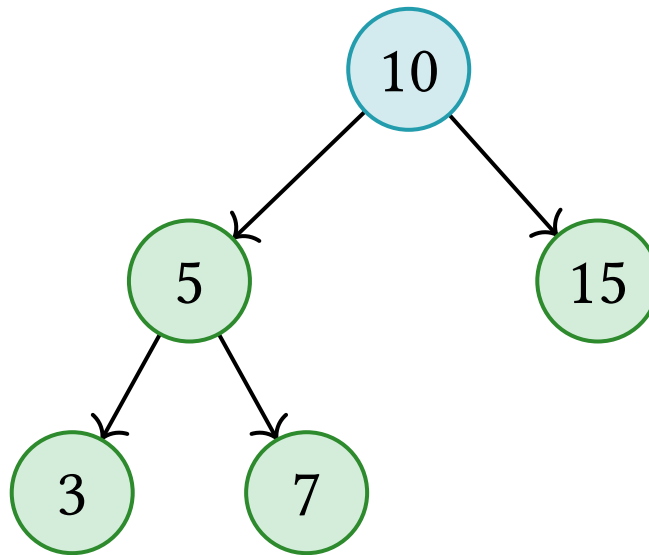
Postorder na stromu [10, 5, 15, 3, 7]:



Výstup: 3, 7, 5, 15

Postorder (LRN)

Postorder na stromu [10, 5, 15, 3, 7]:



Výstup: 3, 7, 5, 15, 10

Složitost průchodů

Složitost průchodů

Kolik uzlů musíme navštívit při průchodu celým stromem?

Složitost průchodů

Kolik uzlů musíme navštívit při průchodu celým stromem?

Všechny $\Rightarrow$ složitost je

Složitost průchodů

Kolik uzlů musíme navštívit při průchodu celým stromem?

Všechny $\Rightarrow$ složitost je $\mathcal{O}(n)$ pro všechny tři průchody

Složitost průchodů

Kolik uzlů musíme navštívit při průchodu celým stromem?

Všechny $\Rightarrow$ složitost je $\mathcal{O}(n)$ pro všechny tři průchody

A kolik paměti spotřebujeme?

Složitost průchodů

Kolik uzlů musíme navštívit při průchodu celým stromem?

Všechny $\Rightarrow$ složitost je $\mathcal{O}(n)$ pro všechny tři průchody

A kolik paměti spotřebujeme? Stack framy na zásobníku, takže záleží na **výšce** stromu

Složitost průchodů

Kolik uzlů musíme navštívit při průchodu celým stromem?

Všechny $\Rightarrow$ složitost je $\mathcal{O}(n)$ pro všechny tři průchody

A kolik paměti spotřebujeme? Stack framy na zásobníku, takže záleží na **výšce** stromu

- Vyvážený strom:

Složitost průchodů

Kolik uzlů musíme navštívit při průchodu celým stromem?

Všechny $\Rightarrow$ složitost je $\mathcal{O}(n)$ pro všechny tři průchody

A kolik paměti spotřebujeme? Stack framy na zásobníku, takže záleží na **výšce** stromu

- Vyvážený strom: $\mathcal{O}(\log n)$

Složitost průchodů

Kolik uzlů musíme navštívit při průchodu celým stromem?

Všechny $\Rightarrow$ složitost je $\mathcal{O}(n)$ pro všechny tři průchody

A kolik paměti spotřebujeme? Stack framy na zásobníku, takže záleží na **výšce** stromu

- Vyvážený strom: $\mathcal{O}(\log n)$
- Degenerovaný strom (ukážeme později):

Složitost průchodů

Kolik uzlů musíme navštívit při průchodu celým stromem?

Všechny $\Rightarrow$ složitost je $\mathcal{O}(n)$ pro všechny tři průchody

A kolik paměti spotřebujeme? Stack framy na zásobníku, takže záleží na **výšce** stromu

- Vyvážený strom: $\mathcal{O}(\log n)$
- Degenerovaný strom (ukážeme později): $\mathcal{O}(n)$

Uvolnění stromu z paměti

Jak byste takto vytvořenou datovou strukturu uvolnili z paměti?

Uvolnění stromu z paměti

Jak byste takto vytvořenou datovou strukturu uvolnili z paměti?

Nemůžeme uvolnit uzel **před** jeho potomky, protože bychom

Uvolnění stromu z paměti

Jak byste takto vytvořenou datovou strukturu uvolnili z paměti?

Nemůžeme uvolnit uzel **před** jeho potomky, protože bychom k nim ztratili přístup

Uvolnění stromu z paměti

Jak byste takto vytvořenou datovou strukturu uvolnili z paměti?

Nemůžeme uvolnit uzel **před** jeho potomky, protože bychom k nim ztratili přístup

```
void free_tree(Node *node) {  
    if (node == NULL) return;  
    free_tree(node->left);  
    free_tree(node->right);  
    free(node);  
}
```

Uvolnění stromu z paměti

Pozor: kdybychom zavolali `free(node)` **před** `free_tree(node->left)`,
přistupovali bychom

Uvolnění stromu z paměti

Pozor: kdybychom zavolali `free(node)` **před** `free_tree(node->left)`,
přistupovali bychom k již uvolněné paměti $\Rightarrow$ **use after free!**

Uvolnění stromu z paměti

Pozor: kdybychom zavolali `free(node)` **před** `free_tree(node->left)`, přistupovali bychom k již uvolněné paměti $\Rightarrow$ **use after free!**

Všimněte si, že `free_tree` vlastně odpovídá **postorder** průchodu – nejprve zpracujeme oba podstromy a teprve pak aktuální uzel

Shrnutí

Shrnutí

Shrnutí

- O **binárním stromu** mluvíme, když každý uzel má maximálně 2 potomky

Shrnutí

- O **binárním stromu** mluvíme, když každý uzel má maximálně 2 potomky
- Typy: plný (*full*), úplný (*complete*), perfektní (*perfect*)

Shrnutí

- O **binárním stromu** mluvíme, když každý uzel má maximálně 2 potomky
- Typy: plný (*full*), úplný (*complete*), perfektní (*perfect*)
- V C implementujeme uzlem se dvěma ukazateli: `left` a `right`

Shrnutí

- O **binárním stromu** mluvíme, když každý uzel má maximálně 2 potomky
- Typy: plný (*full*), úplný (*complete*), perfektní (*perfect*)
- V C implementujeme uzlem se dvěma ukazateli: `left` a `right`
- Tři průchody: **inorder** (LNR), **preorder** (NLR), **postorder** (LRN)

Shrnutí

- O **binárním stromu** mluvíme, když každý uzel má maximálně 2 potomky
- Typy: plný (*full*), úplný (*complete*), perfektní (*perfect*)
- V C implementujeme uzlem se dvěma ukazateli: `left` a `right`
- Tři průchody: **inorder** (LNR), **preorder** (NLR), **postorder** (LRN)
- Všechny průchody mají časovou složitost $\mathcal{O}(n)$

Shrnutí

- O **binárním stromu** mluvíme, když každý uzel má maximálně 2 potomky
- Typy: plný (*full*), úplný (*complete*), perfektní (*perfect*)
- V C implementujeme uzlem se dvěma ukazateli: `left` a `right`
- Tři průchody: **inorder** (LNR), **preorder** (NLR), **postorder** (LRN)
- Všechny průchody mají časovou složitost $\mathcal{O}(n)$
- Paměťová složitost závisí na výšce stromu: $\mathcal{O}(\log n)$ až $\mathcal{O}(n)$